

CS120 Projects' Report

Bai Yi Chun

Lu Ao

January 15, 2025

1 ABSTRACT

Throughout this semester, we dedicated several weeks to working on this Computer Network project, which, though challenging, proved to be a rewarding experience. By the end, we successfully completed nearly all of the basic tasks. Many of these tasks were difficult to grasp at first, but the insights and experiences shared by previous students were invaluable in helping us navigate through them. That's why we decide to write our own report, concluding the challenges we faced in this project, and hope that could help others who want to catch up in this course.

2 PROJ0

The **Proj0** part is relatively straightforward, primarily focusing on installing necessary tools and running simple examples. For those new to the project, we recommend exploring the ASIO library in greater depth during Proj0. This will provide a solid foundation for tackling more complex tasks in later projects.

One key point to mention is that, instead of using the suggested Visual Studio, we opted to use CMake with VSCode to set up the coding environment. This approach simplifies the process and avoids the complexity often associated with Visual Studio. We encourage everyone to try this setup, as it only requires basic knowledge of the ASIO library and CMake. Of course, feel free to reach out for our help if needed.

3 PROJ1

Finally, we began working on the first real project, and this is where things started to get both interesting and challenging (**or, some might say, terrible**). Some even consider **Proj1** to be the most difficult part, especially when compared to the subsequent projects. Although we didn't spend as much time on Proj1 as we did on Proj2, we still found it to be time-consuming and frustrating due to its inherent complexity. As a reference, we spent nearly 3-4 full days completing this project.

3.1 CHALLENGES

In Project 1, we were tasked with establishing a data link between two air-gapped computer nodes by precisely manipulating and sensing sound signals. This project presented several challenges, particularly in signal modulation and preamble detection. One key requirement was to accurately detect a series of target signals before beginning the demodulation process. We spent nearly an entire day trying to get our preamble detection to work correctly.

Another particularly frustrating (perhaps the most frustrating) aspect was the hardware. In Project 1, we were required to use only our own equipment for signal transmission, and the performance of our hardware had a significant impact on the overall performance of the code. Additionally, the physical environment played a major role in the results, meaning we had to continuously adjust our code to accommodate changes in our surroundings each time we moved to a new location. This part was extremely time-consuming and nearly drove us crazy during testing.

3.2 SUGGESTIONS

1. **Understand and fully utilize the provided code example**—they can be a lifesaver. The example covers most of the implementation, from modulation and demodulation to preamble detection. Be sure to understand the code and implement it yourself; simply copying it is likely to result in poor performance.
2. **Use high-quality audio equipment.** We used high-end gaming headphones to play the sound and Apple AirPods to record. Make sure to adjust the volume properly; generally, a higher volume yields better results.
3. **Visualize your signal** to check whether it behaves as expected. Don't skip this step out of laziness (it's quite easy to do using Python). Visualizing your signal is incredibly helpful because it reveals its true shape. For example, you can quickly spot if the volume is too low or too high. Additionally, it plays a crucial role in debugging the preamble detection.

4 PROJ2

After completing Proj1, you'll have a clearer understanding of the true challenges of this course. Now, it's time to tackle Proj2. Our first piece of advice is to start as early as possible, as this project took us 5 to 6 full days to complete. In Proj2, you'll begin working with the audio cards

and other equipment provided by the TAs. It's a good idea to first test the performance of these tools before diving into the project.

4.1 CHALLENGES

The first challenge we encountered was how to remodulate the bits. In this project, we have a larger volume of bits to transmit, and the TAs will monitor the total time you take. As a result, a simplified modulation method is required. Additionally, the transmitted bits often consist of long sequences of consecutive 0s or 1s. These sequences cause the signal to remain unchanged for extended periods, which can overwhelm the hardware. It's crucial to carefully manage the signal before moving on to the later tasks.

Next, we tackled the MAC layer, which we found to be the most code-intensive part of the project. We implemented the state transitions using a standard FSM, but there may be simpler alternatives, such as using asynchronous methods, to achieve the same result.

Finally, handling packet collisions proved to be another challenging aspect, requiring extensive testing and debugging to resolve effectively.

4.2 SUGGESTIONS

1. **Use Square Wave modulation.** This method works well since now we're using cables to transmit the signal, which provides much more stable transmission.
2. **Visualize the signal.** This step is still crucial for debugging, as it helps identify potential issues with the signal's behavior.
3. **Adjust the frequency or sample rate** of your signal if the transmission is taking too long. This can help optimize performance.
4. **Review relevant concepts**—especially those covered in previous lectures before starting the coding phase. A solid understanding of the theory will make the implementation smoother.

5 PROJ3

Previous project focused on the physical and data link layers, while **Proj3** shifted the focus to the network layer. The key point of this project is to implement the sending and receiving of IP packets, including sending and receiving ICMP packets, routing, and NAT.

5.1 CHALLENGES

In terms of ICMP packets, we encountered some difficulties when implementing the echo request and reply. Unfortunately, we found that the packet we sent was the same as the expected packet but we did not receive the reply. After debugging, we found that the problem was caused by the checksum of the ICMP packet. We calculated the checksum of the ICMP packet incorrectly (we directly set the checksum to 0), which caused the packet to be discarded.

by the receiver. After fixing the checksum calculation, we successfully received the echo reply. We guess that the reason why Wireshark captured the true packet with checksum = 0 is that Wireshark may have some mechanism to ignore the checksum of the packet. In practice, we should always calculate the checksum of the packet correctly to avoid the packet being discarded by the receiver.

5.2 SUGGESTIONS

1. **Use WireShark to capture and analyze real packets.** It's essential to understand the structure of the ICMP packet, including the header and payload. WireShark is a useful tool for capturing and analyzing packets, which can help you understand the structure of the packets. When meeting problems, we can use WireShark to capture the packets which sent by npcap and compare them with the expected packets.
2. **Figure out what to do with the source ip address and destination ip address for each node.** It's easy for us since we did not finish the optional part of the project then the transmitting path is fixed. What's more, we can just save a static NAT table.

6 PROJ4

Above IP layer, we have the transport layer, which is the focus of **Proj4**. The main goal of this project is to implement the DNS protocol, which is based on UDP. DNS is used to resolve domain names to IP addresses, and it is a critical part of the Internet. After obtaining the ip, we can use it to send the ICMP packet to the destination which is implemented in Proj3.

6.1 CHALLENGES

Obviously, DNS is more complicated than ICMP. When we use cmd to ping a domain name, we can capture several DNS query packets, including both ipv4 and ipv6 type. And the response packets are even more complex, which contain several answers with various types. We need to parse the DNS packet correctly to get the IP address of the target domain name. It's necessary to figure out which part of the packet is the header, which part is the question, and which part is the answer. In the answer part, we need to figure out the type of the answer and the length of the answer. We should traverse the answer part to get the IP address of the target domain name with the correct type. Using Wireshark to analyze each offset of the packet is a good way to understand the structure of the packet.

6.2 SUGGESTIONS

1. **Ignore dns checksum.** We found that the checksum of the DNS packet is not necessary. We suggest that just set the checksum to 0 and the packet can be received correctly. Since the calculation of the dns checksum is not clear. We tried several generations of the checksum but none of them worked. It contains the pseudo header and even some padding which makes the calculation more difficult.